CHAPTER

7 REQUIREMENTS MODELING: FLOW, BEHAVIOR, PATTERNS, AND WEBAPPS

KEY CONCEPTS

After my discussion of use cases, data modeling, and class-based models in Chapter 6, it's reasonable to ask, "Aren't those requirements modeling representations enough?"

The only reasonable answer is, "That depends."

For some types of software, the use case may be the only requirements modeling representation that is required. For others, an object-oriented approach is chosen and class-based models may be developed. But in other situations, complex application requirements may demand an examination of how data objects are transformed as they move through a system; how an application behaves as a consequence of external events; whether existing domain knowledge can be adapted to the current problem; or in the case of Web-based systems and applications, how content and functionality meld to provide an end user with the ability to successfully navigate a WebApp to achieve usage goals.

7.1 REQUIREMENTS MODELING STRATEGIES

One view of requirements modeling, called *structured analysis,* considers data and the processes that transform the data as separate entities. Data objects are modeled in a way that defines their attributes and relationships. Processes that manipulate data objects are modeled in a manner that shows how they transform data as data objects flow through the system. A second approach to analysis

QUICK LOOK

What is it? The requirements model has many different dimensions. In this chapter you'll learn about flow-oriented models, behavioral models, and the special requirements analysis considerations that come into play when WebApps are developed. Each of these modeling representations supplements the use cases, data models, and class-based models discussed in Chapter 6.

Who does it? A software engineer (sometimes called an "analyst") builds the model using requirements elicited from various stakeholders.

Why is it important? Your insight into software requirements grows in direct proportion to the number of different requirements modeling dimensions. Although you may not have the time, the resources, or the inclination to develop every representation suggested in this chapter and Chapter 6, recognize that each different modeling approach provides you with a different way of looking at the problem. As a consequence, you (and other stakeholders) will be better able to assess whether you've properly specified what must be accomplished.

What are the steps? Flow-oriented modeling provides an indication of how data objects are transformed by processing functions. Behavioral modeling depicts the states of the system and its classes and the impact of events on these states. Pattern-based modeling makes use of existing domain knowledge to facilitate requirements analysis. WebApp requirements models are especially adapted for the representation of content, interaction, function, and configuration-related requirements.

What is the work product? A wide array of text-based and diagrammatic forms may be chosen for the requirements model. Each of these representations provides a view of one or more of the model elements.

How do I ensure that I've done it right? Requirements modeling work products must be reviewed for correctness, completeness, and consistency. They must reflect the needs of all stakeholders and establish a foundation from which design can be conducted.

modeled, called *object-oriented analysis,* focuses on the definition of classes and the manner in which they collaborate with one another to effect customer requirements.

Although the analysis model that we propose in this book combines features of both approaches, software teams often choose one approach and exclude all representations from the other. The question is not which is best, but rather, what combination of representations will provide stakeholders with the best model of software requirements and the most effective bridge to software design.

7.2 Flow-Oriented Modeling

Although data flow-oriented modeling is perceived as an outdated technique by some software engineers, it continues to be one of the most widely used requirements analysis notations in use today.[1] Although the *data flow diagram* (DFD) and related diagrams and information are not a formal part of UML, they can be used to complement UML diagrams and provide additional insight into system requirements and flow.

Some will suggest that the DFD is old-school and it has no place in modern practice. That's a view that excludes a potentially useful mode of representation at the analysis level. If it can help, use the DFD.

The DFD takes an input-process-output view of a system. That is, data objects flow into the software, are transformed by processing elements, and resultant data objects flow out of the software. Data objects are represented by labeled arrows, and transformations are represented by circles (also called bubbles). The DFD is presented in a hierarchical fashion. That is, the first data flow model (sometimes called a level 0 DFD or *context diagram*) represents the system as a whole. Subsequent data flow diagrams refine the context diagram, providing increasing detail with each subsequent level.

1 Data flow modeling is a core modeling activity in *structured analysis.*

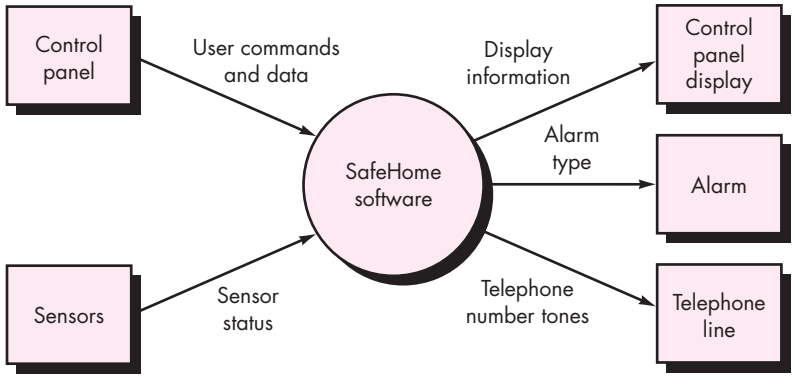


FIGURE 7.1 Context-level DFD for the *SafeHome* security function

7.2.1 Creating a Data Flow Model

The data flow diagram enables you to develop models of the information domain and functional domain. As the DFD is refined into greater levels of detail, you perform an implicit functional decomposition of the system. At the same time, the DFD refinement results in a corresponding refinement of data as it moves through the processes that embody the application.

A few simple guidelines can aid immeasurably during the derivation of a data flow diagram: (1) the level 0 data flow diagram should depict the software/system as a single bubble; (2) primary input and output should be carefully noted; (3) refinement should begin by isolating candidate processes, data objects, and data stores to be represented at the next level; (4) all arrows and bubbles should be labeled with meaningful names; (5) *information flow continuity* must be maintained from level to level,[2] and (6) one bubble at a time should be refined. There is a natural tendency to overcomplicate the data flow diagram. This occurs when you attempt to show too much detail too early or represent procedural aspects of the software in lieu of information flow.

To illustrate the use of the DFD and related notation, we again consider the *SafeHome* security function. A level 0 DFD for the security function is shown in Figure 7.1. The primary *external entities* (boxes) produce information for use by the system and consume information generated by the system. The labeled arrows represent data objects or data object hierarchies. For example, **user commands and data** encompasses all configuration commands, all activation/deactivation commands, all miscellaneous interactions, and all data that are entered to qualify or expand a command.

The level 0 DFD must now be expanded into a level 1 data flow model. But how do we proceed? Following an approach suggested in Chapter 6, you should apply a

Quote:

"The purpose of data flow diagrams is to provide a semantic bridge between users and systems developers."

Kenneth Kozar

KEY POINT

Information flow continuity must be maintained as each DFD level is refined. This means that input and output at one level must be the same as input and output at a refined level.

2 That is, the data objects that flow into the system or into any transformation at one level must be the same data objects (or their constituent parts) that flow into the transformation at a more refined level.

"grammatical parse" [Abb83] to the use case narrative that describes the context-level bubble. That is, we isolate all nouns (and noun phrases) and verbs (and verb phrases) in a *SafeHome* processing narrative derived during the first requirements gathering meeting. Recalling the parsed processing narrative text presented in Section 6.5.1:

The grammatical parse is not foolproof, but it can provide you with an excellent jump start, if you're struggling to define data objects and the transforms that operate on them.

> The SafeHome security function *enables* the homeowner to *configure* the security system when it is *installed, monitors* all sensors *connected* to the security system, and *interacts* with the homeowner through the Internet, a PC, or a control panel.
>
> During installation, the *SafeHome* PC is used to *program* and *configure* the system. Each sensor is assigned a number and type, a master password is programmed for *arming* and *disarming* the system, and telephone number(s) are *input* for *dialing* when a sensor event occurs.
>
> When a sensor event is *recognized*, the software *invokes* an audible alarm attached to the system. After a delay time that is specified by the homeowner during system configuration activities, the software dials a telephone number of a monitoring service, *provides* information about the location, *reporting* the nature of the event that has been detected. The telephone number will be *redialed* every 20 seconds until telephone connection is *obtained.*
>
> The homeowner *receives* security information via a control panel, the PC, or a browser, collectively called an interface. The interface *displays* prompting messages and system status information on the control panel, the PC, or the browser window. Homeowner interaction takes the following form . . .

Be certain that the processing narrative you intend to parse is written at the same level of abstraction throughout.

Referring to the grammatical parse, verbs are *SafeHome* processes and can be represented as bubbles in a subsequent DFD. Nouns are either external entities (boxes), data or control objects (arrows), or data stores (double lines). From the discussion in Chapter 6, recall that nouns and verbs can be associated with one another (e.g., each sensor is assigned a number and type; therefore **number** and **type** are attributes of the data object **sensor**). Therefore, by performing a grammatical parse on the processing narrative for a bubble at any DFD level, you can generate much useful information about how to proceed with the refinement to the next level. Using this information, a level 1 DFD is shown in Figure 7.2. The context level process shown in Figure 7.1 has been expanded into six processes derived from an examination of the grammatical parse. Similarly, the information flow between processes at level 1 has been derived from the parse. In addition, information flow continuity is maintained between levels 0 and 1.

The processes represented at DFD level 1 can be further refined into lower levels. For example, the process *monitor sensors* can be refined into a level 2 DFD as shown in Figure 7.3. Note once again that information flow continuity has been maintained between levels.

The refinement of DFDs continues until each bubble performs a simple function. That is, until the process represented by the bubble performs a function that would be easily implemented as a program component. In Chapter 8, I discuss a concept, called *cohesion,* that can be used to assess the processing focus of a given function. For now, we strive to refine DFDs until each bubble is "single-minded."

FIGURE 7.2

Level 1 DFD for *SafeHome* security function

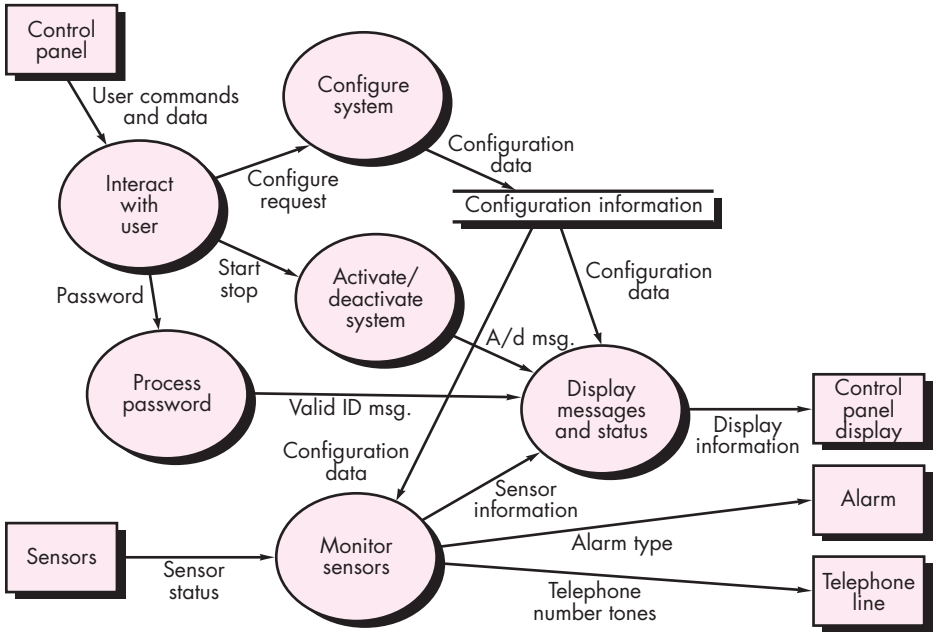
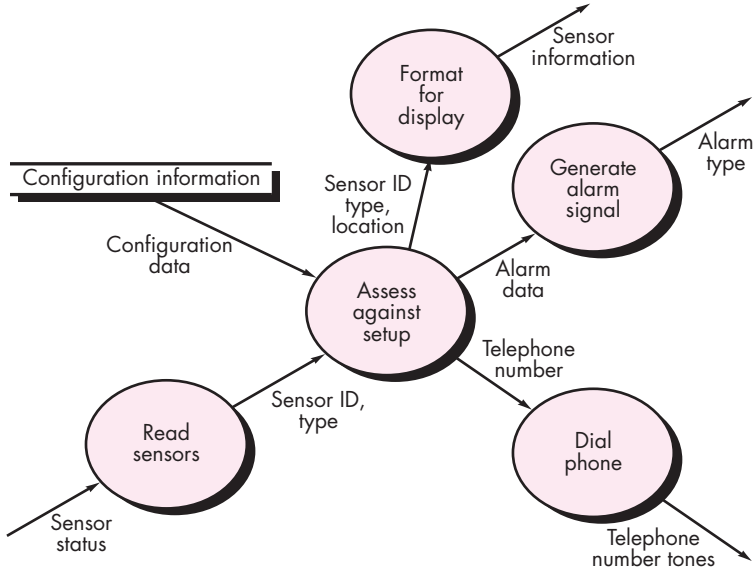


FIGURE 7.3

Level 2 DFD that refines the *monitor sensors* process



7.2.2 Creating a Control Flow Model

For some types of applications, the data model and the data flow diagram are all that is necessary to obtain meaningful insight into software requirements. As I have already noted, however, a large class of applications are "driven" by events rather than data, produce control information rather than reports or displays, and process information with heavy concern for time and performance. Such applications require the use of *control flow modeling* in addition to data flow modeling.

I have already noted that an event or control item is implemented as a Boolean value (e.g., true or false, on or off, 1 or 0) or a discrete list of conditions (e.g., empty, jammed, full). To select potential candidate events, the following guidelines are suggested:

? How do I select potential events for a control flow diagram, state diagram, or CSPEC?

- List all sensors that are "read" by the software.
- List all interrupt conditions.
- List all "switches" that are actuated by an operator.
- List all data conditions.
- Recalling the noun/verb parse that was applied to the processing narrative, review all "control items" as possible control specification inputs/outputs.
- Describe the behavior of a system by identifying its states, identify how each state is reached, and define the transitions between states.
- Focus on possible omissions—a very common error in specifying control; for example, ask: "Is there any other way I can get to this state or exit from it?"

Among the many events and control items that are part of *SafeHome* software are **sensor event** (i.e., a sensor has been tripped), **blink flag** (a signal to blink the display), and **start/stop switch** (a signal to turn the system on or off).

7.2.3 The Control Specification

A *control specification* (CSPEC) represents the behavior of the system (at the level from which it has been referenced) in two different ways.[3] The CSPEC contains a state diagram that is a sequential specification of behavior. It can also contain a program activation table—a combinatorial specification of behavior.

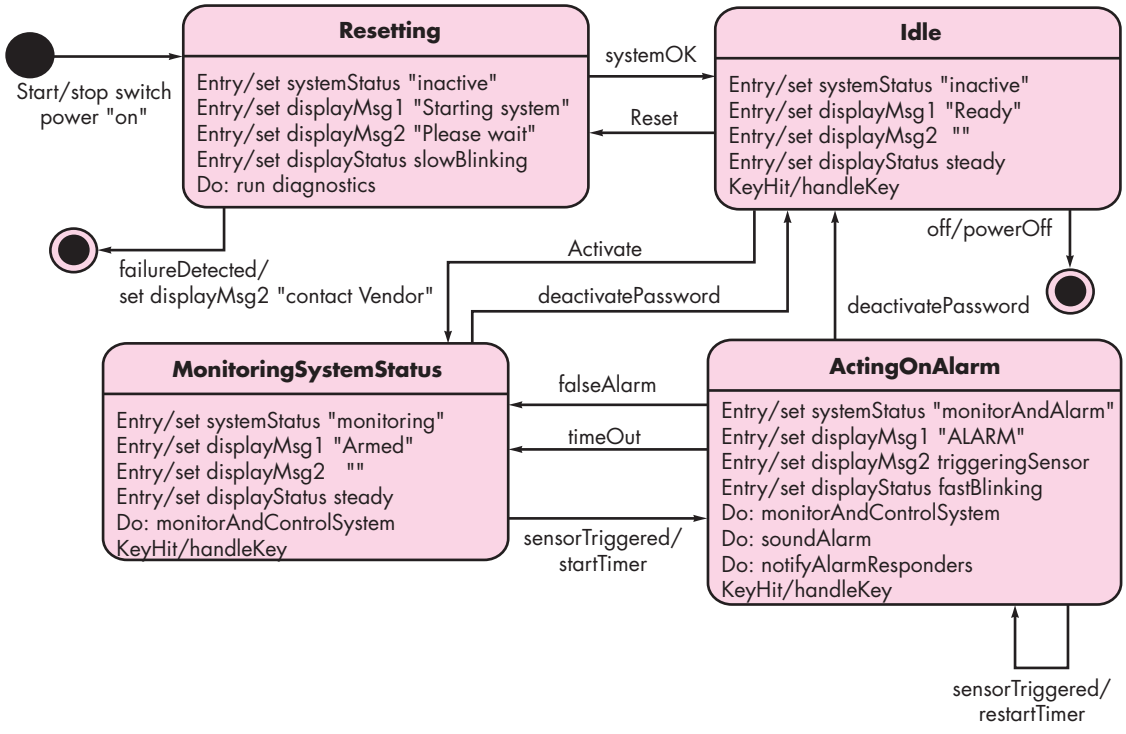
Figure 7.4 depicts a preliminary state diagram[4] for the level 1 control flow model for *SafeHome.* The diagram indicates how the system responds to events as it traverses the four states defined at this level. By reviewing the state diagram, you can determine the behavior of the system and, more important, ascertain whether there are "holes" in the specified behavior.

For example, the state diagram (Figure 7.4) indicates that the transitions from the **Idle** state can occur if the system is reset, activated, or powered off. If the system is

3 Additional behavioral modeling notation is presented in Section 7.3.

4 The state diagram notation used here conforms to UML notation. A "state transition diagram" is available in structured analysis, but the UML format is superior in information content and representation.

FIGURE 7.4 State diagram for *SafeHome* security function



activated (i.e., alarm system is turned on), a transition to the **MonitoringSystemStatus** state occurs, display messages are changed as shown, and the process *monitorAndControlSystem* is invoked. Two transitions occur out of the **MonitoringSystemStatus** state—(1) when the system is deactivated, a transition occurs back to the **Idle** state; (2) when a sensor is triggered into the **ActingOnAlarm** state. All transitions and the content of all states are considered during the review.

A somewhat different mode of behavioral representation is the process activation table. The PAT represents information contained in the state diagram in the context of processes, not states. That is, the table indicates which processes (bubbles) in the flow model will be invoked when an event occurs. The PAT can be used as a guide for a designer who must build an executive that controls the processes represented at this level. A PAT for the level 1 flow model of *SafeHome* software is shown in Figure 7.5.

The CSPEC describes the behavior of the system, but it gives us no information about the inner working of the processes that are activated as a result of this behavior. The modeling notation that provides this information is discussed in Section 7.2.4.

7.2.4 The Process Specification

The *process specification* (PSPEC) is used to describe all flow model processes that appear at the final level of refinement. The content of the process specification can

FIGURE 7.5

Process activation table for *SafeHome* security function

input events						
sensor event	0	0	0	0	1	0
blink flag	0	0	1	1	0	0
start stop switch	0	1	0	0	0	0
display action status complete	0	0	0	1	0	0
in-progress	0	0	1	0	0	0
time out	0	0	0	0	0	1
output						
alarm signal	0	0	0	0	1	0
process activation						
monitor and control system	0	1	0	0	1	1
activate/deactivate system	0	1	0	0	0	0
display messages and status	1	0	1	1	1	1
interact with user	1	0	0	1	0	1

SAFEHOME

Data Flow Modeling

The scene: Jamie's cubicle, after the last requirements gathering meeting has concluded.

The players: Jamie, Vinod, and Ed—all members of the *SafeHome* software engineering team.

The conversation:

(Jamie has sketched out the models shown in Figures 7.1 through 7.5 and is showing them to Ed and Vinod.)

Jamie: I took a software engineering course in college, and they taught us this stuff. The Prof said it's a bit old-fashioned, but you know what, it helps me to clarify things.

Ed: That's cool. But I don't see any classes or objects here.

Jamie: No . . . this is just a flow model with a little behavioral stuff thrown in.

Vinod: So these DFDs represent an I-P-O view of the software, right.

Ed: I-P-O?

Vinod: Input-process-output. The DFDs are actually pretty intuitive . . . if you look at 'em for a moment, they show how data objects flow through the system and get transformed as they go.

Ed: Looks like we could convert every bubble into an executable component . . . at least at the lowest level of the DFD.

Jamie: That's the cool part, you can. In fact, there's a way to translate the DFDs into an design architecture.

Ed: Really?

Jamie: Yeah, but first we've got to develop a complete requirements model and this isn't it.

Vinod: Well, it's a first step, but we're going to have to address class-based elements and also behavioral aspects, although the state diagram and PAT does some of that.

Ed: We've got a lot work to do and not much time to do it.

(Doug—the software engineering manager—walks into the cubical.)

Doug: So the next few days will be spent developing the requirements model, huh?

Jamie (looking proud): We've already begun.

Doug: Good, we've got a lot of work to do and not much time to do it.

(The three software engineers look at one another and smile.)

KEY POINT
The PSPEC is a "mini-specification" for each transform at the lowest refined level of a DFD.

include narrative text, a program design language (PDL) description[5] of the process algorithm, mathematical equations, tables, or UML activity diagrams. By providing a PSPEC to accompany each bubble in the flow model, you can create a "mini-spec" that serves as a guide for design of the software component that will implement the bubble.

To illustrate the use of the PSPEC, consider the *process password* transform represented in the flow model for *SafeHome* (Figure 7.2). The PSPEC for this function might take the form:

> **PSPEC: process password (at control panel).** The *process password* transform performs password validation at the control panel for the *SafeHome* security function. *Process password* receives a four-digit password from the *interact with user* function. The password is first compared to the master password stored within the system. If the master password matches, <valid id message = true> is passed to the *message and status display* function. If the master password does not match, the four digits are compared to a table of secondary passwords (these may be assigned to house guests and/or workers who require entry to the home when the owner is not present). If the password matches an entry within the table, <valid id message = true> is passed to the *message and status display function*. If there is no match, <valid id message = false> is passed to the message and status display function.

If additional algorithmic detail is desired at this stage, a program design language representation may also be included as part of the PSPEC. However, many believe that the PDL version should be postponed until component design commences.

SOFTWARE TOOLS

Structured Analysis

Objective: Structured analysis tools allow a software engineer to create data models, flow models, and behavioral models in a manner that enables consistency and continuity checking and easy editing and extension. Models created using these tools provide the software engineer with insight into the analysis representation and help to eliminate errors before they propagate into design, or worse, into implementation itself.

Mechanics: Tools in this category use a "data dictionary" as the central database for the description of all data objects. Once entries in the dictionary are defined, entity-relationship diagrams can be created and object hierarchies can be developed. Data flow diagramming features allow easy creation of this graphical model and also provide features for the creation of PSPECs and CSPECs. Analysis tools also enable the software engineer to create behavioral models using the state diagram as the operative notation.

Representative Tools:[6]

MacA&D, WinA&D, developed by Excel software (**www.excelsoftware.com**), provides a set of simple and inexpensive analysis and design tools for Macs and Windows machines.

MetaCASE Workbench, developed by MetaCase Consulting (**www.metacase.com**), is a metatool used to define an analysis or design method (including structured analysis) and its concepts, rules, notations, and generators.

System Architect, developed by Popkin Software (**www.popkin.com**) provides a broad range of analysis and design tools including tools for data modeling and structured analysis.

5 Program design language (PDL) mixes programming language syntax with narrative text to provide procedural design detail. PDL is discussed briefly in Chapter 10.

6 Tools noted here do not represent an endorsement, but rather a sampling of tools in this category. In most cases, tool names are trademarked by their respective developers.

7.3 Creating a Behavioral Model

How do I model the software's reaction to some external event?

The modeling notation that I have discussed to this point represents static elements of the requirements model. It is now time to make a transition to the dynamic behavior of the system or product. To accomplish this, you can represent the behavior of the system as a function of specific events and time.

The *behavioral model* indicates how software will respond to external events or stimuli. To create the model, you should perform the following steps:

1. Evaluate all use cases to fully understand the sequence of interaction within the system.
2. Identify events that drive the interaction sequence and understand how these events relate to specific objects.
3. Create a sequence for each use case.
4. Build a state diagram for the system.
5. Review the behavioral model to verify accuracy and consistency.

Each of these steps is discussed in the sections that follow.

7.3.1 Identifying Events with the Use Case

In Chapter 6 you learned that the use case represents a sequence of activities that involves actors and the system. In general, an event occurs whenever the system and an actor exchange information. In Section 7.2.3, I indicated that an event is *not* the information that has been exchanged, but rather the fact that information has been exchanged.

A use case is examined for points of information exchange. To illustrate, we reconsider the use case for a portion of the *SafeHome* security function.

> The homeowner uses the keypad to key in a four-digit password. The password is compared with the valid password stored in the system. If the password is incorrect, the control panel will beep once and reset itself for additional input. If the password is correct, the control panel awaits further action.

The underlined portions of the use case scenario indicate events. An actor should be identified for each event; the information that is exchanged should be noted, and any conditions or constraints should be listed.

As an example of a typical event, consider the underlined use case phrase "homeowner uses the keypad to key in a four-digit password." In the context of the requirements model, the object, **Homeowner,**[7] transmits an event to the object **ControlPanel.** The event might be called *password entered*. The information

7 In this example, we assume that each user (homeowner) that interacts with *SafeHome* has an identifying password and is therefore a legitimate object.

transferred is the four digits that constitute the password, but this is not an essential part of the behavioral model. It is important to note that some events have an explicit impact on the flow of control of the use case, while others have no direct impact on the flow of control. For example, the event *password entered* does not explicitly change the flow of control of the use case, but the results of the event *password compared* (derived from the interaction "password is compared with the valid password stored in the system") will have an explicit impact on the information and control flow of the *SafeHome* software.

Once all events have been identified, they are allocated to the objects involved. Objects can be responsible for generating events (e.g., **Homeowner** generates the *password entered* event) or recognizing events that have occurred elsewhere (e.g., **ControlPanel** recognizes the binary result of the *password compared* event).

7.3.2 State Representations

In the context of behavioral modeling, two different characterizations of states must be considered: (1) the state of each class as the system performs its function and (2) the state of the system as observed from the outside as the system performs its function.[8]

The system has states that represent specific externally observable behavior; a class has states that represent its behavior as the system performs its functions.

The state of a class takes on both passive and active characteristics [Cha93]. A *passive state* is simply the current status of all of an object's attributes. For example, the passive state of the class **Player** (in the video game application discussed in Chapter 6) would include the current **position** and **orientation** attributes of **Player** as well as other features of **Player** that are relevant to the game (e.g., an attribute that indicates **magic wishes remaining**). The *active state* of an object indicates the current status of the object as it undergoes a continuing transformation or processing. The class **Player** might have the following active states: *moving, at rest, injured, being cured; trapped, lost,* and so forth. An event (sometimes called a *trigger*) must occur to force an object to make a transition from one active state to another.

Two different behavioral representations are discussed in the paragraphs that follow. The first indicates how an individual class changes state based on external events and the second shows the behavior of the software as a function of time.

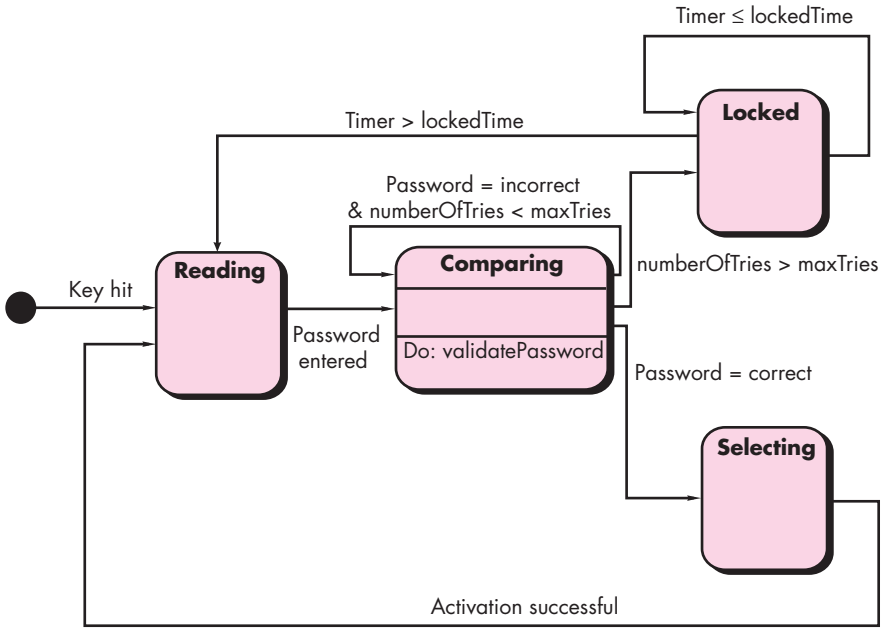
State diagrams for analysis classes. One component of a behavioral model is a UML state diagram[9] that represents active states for each class and the events (triggers) that cause changes between these active states. Figure 7.6 illustrates a state diagram for the **ControlPanel** object in the *SafeHome* security function.

Each arrow shown in Figure 7.6 represents a transition from one active state of an object to another. The labels shown for each arrow represent the event that

8 The state diagrams presented in Chapter 6 and in Section 7.3.2 depict the state of the system. Our discussion in this section will focus on the state of each class within the analysis model.

9 If you are unfamiliar with UML, a brief introduction to this important modeling notation is presented in Appendix 1.

FIGURE 7.6
State diagram for the ControlPanel class



triggers the transition. Although the active state model provides useful insight into the "life history" of an object, it is possible to specify additional information to provide more depth in understanding the behavior of an object. In addition to specifying the event that causes the transition to occur, you can specify a guard and an action [Cha93]. A *guard* is a Boolean condition that must be satisfied in order for the transition to occur. For example, the guard for the transition from the "reading" state to the "comparing" state in Figure 7.6 can be determined by examining the use case:

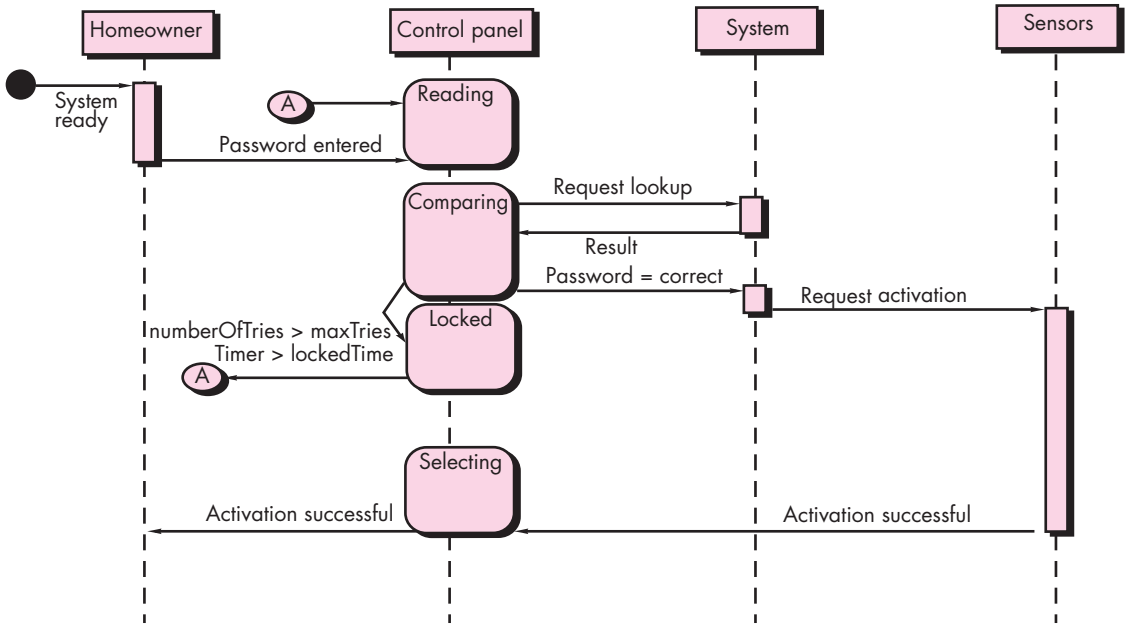
if (password input = 4 digits) then compare to stored password

In general, the guard for a transition usually depends upon the value of one or more attributes of an object. In other words, the guard depends on the passive state of the object.

An *action* occurs concurrently with the state transition or as a consequence of it and generally involves one or more operations (responsibilities) of the object. For example, the action connected to the *password entered* event (Figure 7.6) is an operation named *validatePassword()* that accesses a **password** object and performs a digit-by-digit comparison to validate the entered password.

Sequence diagrams. The second type of behavioral representation, called a *sequence diagram* in UML, indicates how events cause transitions from object to object. Once events have been identified by examining a use case, the modeler

FIGURE 7.7 Sequence diagram (partial) for the *SafeHome* security function



creates a sequence diagram—a representation of how events cause flow from one object to another as a function of time. In essence, the sequence diagram is a shorthand version of the use case. It represents key classes and the events that cause behavior to flow from class to class.

Unlike a state diagram that represents behavior without noting the classes involved, a sequence diagram represents behavior, by describing how classes move from state to state.

Figure 7.7 illustrates a partial sequence diagram for the *SafeHome* security function. Each of the arrows represents an event (derived from a use case) and indicates how the event channels behavior between *SafeHome* objects. Time is measured vertically (downward), and the narrow vertical rectangles represent time spent in processing an activity. States may be shown along a vertical time line.

The first event, *system ready*, is derived from the external environment and channels behavior to the **Homeowner** object. The homeowner enters a password. A *request lookup* event is passed to **System,** which looks up the password in a simple database and returns a *result* (*found* or *not found*) to **ControlPanel** (now in the *comparing* state). A valid password results in a *password=correct* event to **System,** which activates **Sensors** with a *request activation* event. Ultimately, control is passed back to the homeowner with the *activation successful* event.

Once a complete sequence diagram has been developed, all of the events that cause transitions between system objects can be collated into a set of input events and output events (from an object). This information is useful in the creation of an effective design for the system to be built.

SOFTWARE TOOLS

Generalized Analysis Modeling in UML

Objective: Analysis modeling tools provide the capability to develop scenario-based models, class-based models, and behavioral models using UML notation.

Mechanics: Tools in this category support the full range of UML diagrams required to build an analysis model (these tools also support design modeling). In addition to diagramming, tools in this category (1) perform consistency and correctness checks for all UML diagrams, (2) provide links for design and code generation, (3) build a database that enables the management and assessment of large UML models required for complex systems.

Representative Tools:[10]
The following tools support a full range of UML diagrams required for analysis modeling:

ArgoUML is an open source tool available at **argouml.tigris.org**.
Enterprise Architect, developed by Sparx Systems (**www.sparxsystems.com.au**).
PowerDesigner, developed by Sybase (**www.sybase.com**).
Rational Rose, developed by IBM (Rational) (**www01.ibm.com/software/rational/**).
System Architect, developed by Popkin Software (**www.popkin.com**).
UML Studio, developed by Pragsoft Corporation (**www.pragsoft.com**).
Visio, developed by Microsoft (**www.microsoft.com**).
Visual UML, developed by Visual Object Modelers (**www.visualuml.com**).

7.4 PATTERNS FOR REQUIREMENTS MODELING

Software patterns are a mechanism for capturing domain knowledge in a way that allows it to be reapplied when a new problem is encountered. In some cases, the domain knowledge is applied to a new problem within the same application domain. In other cases, the domain knowledge captured by a pattern can be applied by analogy to a completely different application domain.

The original author of an analysis pattern does not "create" the pattern, but, rather, *discovers* it as requirements engineering work is being conducted. Once the pattern has been discovered, it is documented by describing "explicitly the general problem to which the pattern is applicable, the prescribed solution, assumptions and constraints of using the pattern in practice, and often some other information about the pattern, such as the motivation and driving forces for using the pattern, discussion of the pattern's advantages and disadvantages, and references to some known examples of using that pattern in practical applications" [Dev01].

In Chapter 5, I introduced the concept of analysis patterns and indicated that these patterns represent a solution that often incorporates a class, a function, or a behavior within the application domain. The pattern can be reused when performing requirements modeling for an application within a domain.[11] Analysis patterns are stored in a repository so that members of the software team can use search facilities to find and reuse them. Once an appropriate pattern is selected, it is integrated into the requirements model by reference to the pattern name.

10 Tools noted here do not represent an endorsement, but rather a sampling of tools in this category. In most cases, tool names are trademarked by their respective developers.
11 An in-depth discussion of the use of patterns during software design is presented in Chapter 12.

7.4.1 Discovering Analysis Patterns

The requirements model is comprised of a wide variety of elements: scenario-based (use cases), data-oriented (the data model), class-based, flow-oriented, and behavioral. Each of these elements examines the problem from a different perspective, and each provides an opportunity to discover patterns that may occur throughout an application domain, or by analogy, across different application domains.

The most basic element in the description of a requirements model is the use case. In the context of this discussion, a coherent set of use cases may serve as the basis for discovering one or more analysis patterns. A *semantic analysis pattern* (SAP) "is a pattern that describes a small set of coherent use cases that together describe a basic generic application" [Fer00].

Consider the following preliminary use case for software required to control and monitor a real-view camera and proximity sensor for an automobile:

> **Use case: *Monitor reverse motion***
>
> **Description:** When the vehicle is placed in *reverse* gear, the control software enables a video feed from a rear-placed video camera to the dashboard display. The control software superimposes a variety of distance and orientation lines on the dashboard display so that the vehicle operator can maintain orientation as the vehicle moves in reverse. The control software also monitors a proximity sensor to determine whether an object is inside 10 feet of the rear of the vehicle. It will automatically break the vehicle if the proximity sensor indicates an object within *x* feet of the rear of the vehicle, where *x* is determined based on the speed of the vehicle.

This use case implies a variety of functionality that would be refined and elaborated (into a coherent set of use cases) during requirements gathering and modeling. Regardless of how much elaboration is accomplished, the use cases suggest a simple, yet widely applicable SAP—the software-based monitoring and control of sensors and actuators in a physical system. In this case, the "sensors" provide information about proximity and video information. The "actuator" is the breaking system of the vehicle (invoked if an object is very close to the vehicle). But in a more general case, a widely applicable pattern is discovered.

Software in many different application domains is required to monitor sensors and control physical actuators. It follows that an analysis pattern that describes generic requirements for this capability could be used widely. The pattern, called **Actuator-Sensor,** would be applicable as part of the requirements model for *SafeHome* and is discussed in Section 7.4.2, which follows.

7.4.2 A Requirements Pattern Example: Actuator-Sensor[12]

One of the requirements of the *SafeHome* security function is the ability to monitory security sensors (e.g., break-in sensors, fire, smoke or CO sensors, water sensors).

12 This section has been adapted from [Kon02] with the permission of the authors.

Internet-based extensions to *SafeHome* will require the ability to control the movement (e.g., pan, zoom) of a security camera within a residence. The implication—*SafeHome* software must manage various sensors and "actuators" (e.g., camera control mechanisms).

Konrad and Cheng [Kon02] have suggested a requirements pattern named **Actuator-Sensor** that provides useful guidance for modeling this requirement within *SafeHome* software. An abbreviated version of the **Actuator-Sensor** pattern, originally developed for automotive applications, follows.

Pattern Name. **Actuator-Sensor**

Intent. Specify various kinds of sensors and actuators in an embedded system.

Motivation. Embedded systems usually have various kinds of sensors and actuators. These sensors and actuators are all either directly or indirectly connected to a control unit. Although many of the sensors and actuators look quite different, their behavior is similar enough to structure them into a pattern. The pattern shows how to specify the sensors and actuators for a system, including attributes and operations. The **Actuator-Sensor** pattern uses a *pull* mechanism (explicit request for information) for **PassiveSensors** and a *push* mechanism (broadcast of information) for the **ActiveSensors.**

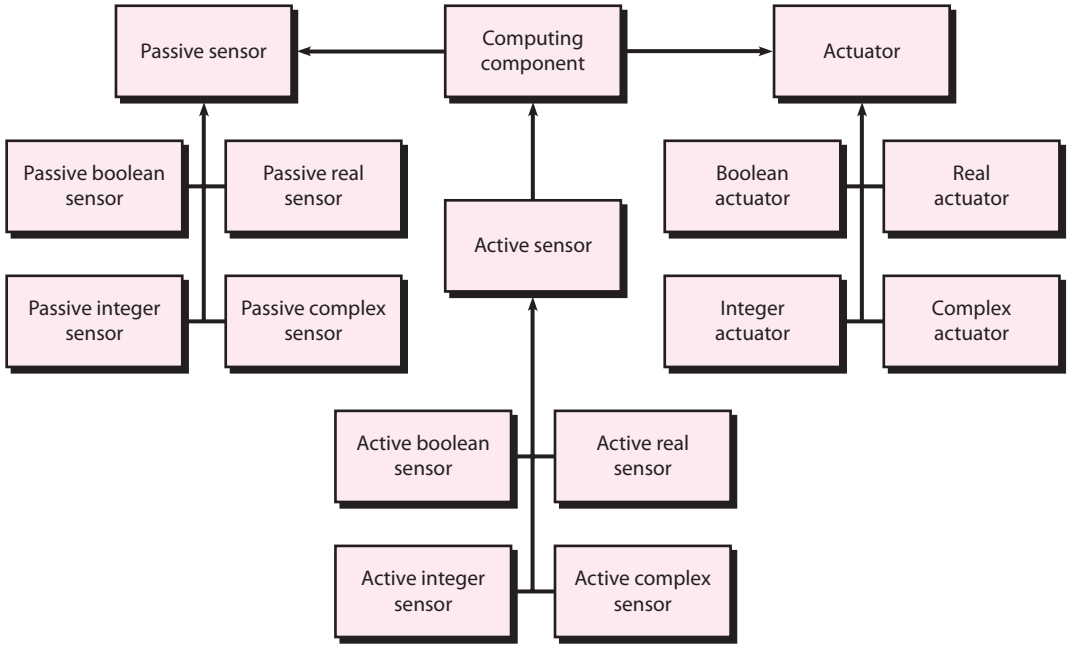
Constraints

- Each passive sensor must have some method to read sensor input and attributes that represent the sensor value.
- Each active sensor must have capabilities to broadcast update messages when its value changes.
- Each active sensor should send a *life tick*, a status message issued within a specified time frame, to detect malfunctions.
- Each actuator must have some method to invoke the appropriate response determined by the **ComputingComponent.**
- Each sensor and actuator should have a function implemented to check its own operation state.
- Each sensor and actuator should be able to test the validity of the values received or sent and set its operation state if the values are outside of the specifications.

Applicability. Useful in any system in which multiple sensors and actuators are present.

Structure. A UML class diagram for the **Actuator-Sensor** pattern is shown in Figure 7.8. **Actuator, PassiveSensor,** and **ActiveSensor** are abstract classes and denoted in italics. There are four different types of sensors and actuators in this pattern.

FIGURE 7.8 UML sequence diagram for the Actuator-Sensor pattern. *Source:* Adapted from [Kon02] with permission.

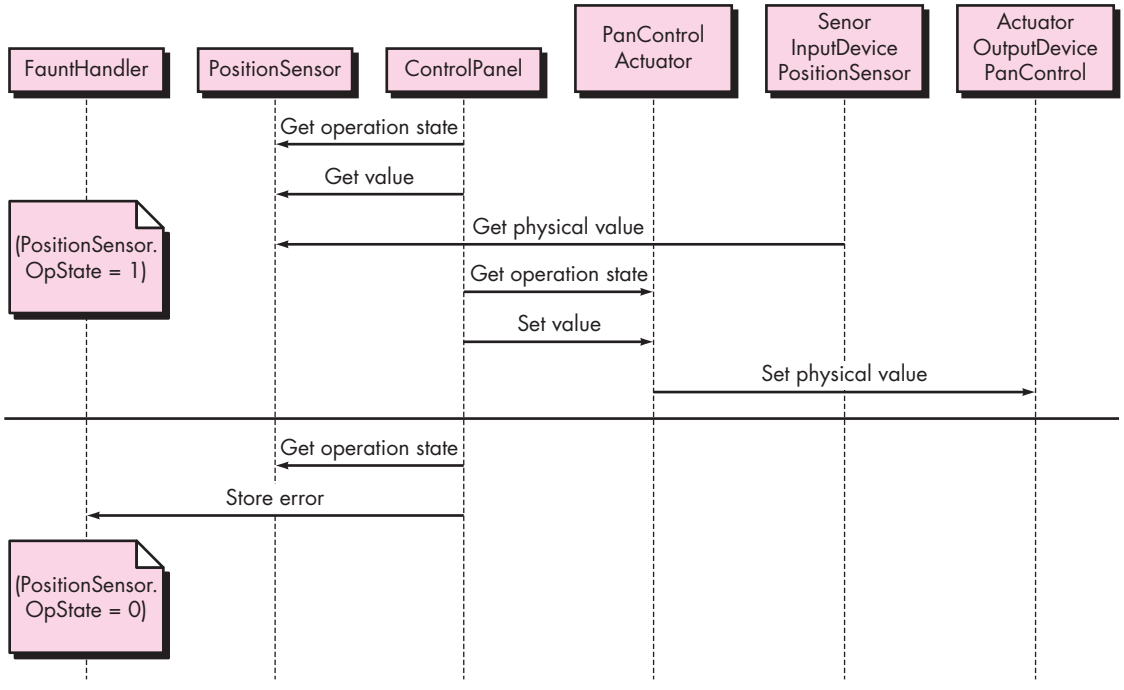


The **Boolean, Integer,** and **Real** classes represent the most common types of sensors and actuators. The complex classes are sensors or actuators that use values that cannot be easily represented in terms of primitive data types, such as a radar device. Nonetheless, these devices should still inherit the interface from the abstract classes since they should have basic functionalities such as querying the operation states.

Behavior. Figure 7.9 presents a UML sequence diagram for an example of the **Actuator-Sensor** pattern as it might be applied for the *SafeHome* function that controls the positioning (e.g., pan, zoom) of a security camera. Here, the **ControlPanel**[13] queries a sensor (a passive position sensor) and an actuator (pan control) to check the operation state for diagnostic purposes before reading or setting a value. The messages *Set Physical Value* and *Get Physical Value* are not messages between objects. Instead, they describe the interaction between the physical devices of the system and their software counterparts. In the lower part of the diagram, below the horizontal line, the **PositionSensor** reports that the operation state is zero. The **ComputingComponent** (represented as **ControlPanel**) then sends the error code for a position sensor failure to the **FaultHandler** that will decide how this error affects the system and what actions are required. It gets the data from the sensors and computes the required response for the actuators.

13 The original pattern uses the generic phrase **ComputingComponent.**

FIGURE 7.9 UML Class diagram for the Actuator-Sensor pattern. *Source:* Reprinted from [Kon02] with permission.



Participants. This section of the patterns description "itemizes the classes/objects that are included in the requirements pattern" [Kon02] and describes the responsibilities of each class/object (Figure 7.8). An abbreviated list follows:

- **PassiveSensor abstract:** Defines an interface for passive sensors.
- **PassiveBooleanSensor:** Defines passive Boolean sensors.
- **PassiveIntegerSensor:** Defines passive integer sensors.
- **PassiveRealSensor:** Defines passive real sensors.
- **ActiveSensor abstract:** Defines an interface for active sensors.
- **ActiveBooleanSensor:** Defines active Boolean sensors.
- **ActiveIntegerSensor:** Defines active integer sensors.
- **ActiveRealSensor:** Defines active real sensors.
- **Actuator abstract:** Defines an interface for actuators.
- **BooleanActuator:** Defines Boolean actuators.
- **IntegerActuator:** Defines integer actuators.
- **RealActuator:** Defines real actuators.
- **ComputingComponent:** The central part of the controller; it gets the data from the sensors and computes the required response for the actuators.

- **ActiveComplexSensor:** Complex active sensors have the basic functionality of the abstract **ActiveSensor** class, but additional, more elaborate, methods and attributes need to be specified.
- **PassiveComplexSensor:** Complex passive sensors have the basic functionality of the abstract **PassiveSensor** class, but additional, more elaborate, methods and attributes need to be specified.
- **ComplexActuator:** Complex actuators also have the base functionality of the abstract **Actuator** class, but additional, more elaborate methods and attributes need to be specified.

Collaborations. This section describes how objects and classes interact with one another and how each carries out its responsibilities.

- When the **ComputingComponent** needs to update the value of a **PassiveSensor,** it queries the sensors, requesting the value by sending the appropriate message.
- **ActiveSensors** are not queried. They initiate the transmission of sensor values to the computing unit, using the appropriate method to set the value in the **ComputingComponent.** They send a life tick at least once during a specified time frame in order to update their timestamps with the system clock's time.
- When the **ComputingComponent** needs to set the value of an actuator, it sends the value to the actuator.
- The **ComputingComponent** can query and set the operation state of the sensors and actuators using the appropriate methods. If an operation state is found to be zero, then the error is sent to the **FaultHandler,** a class that contains methods for handling error messages, such as starting a more elaborate recovery mechanism or a backup device. If no recovery is possible, then the system can only use the last known value for the sensor or the default value.
- The **ActiveSensors** offer methods to add or remove the addresses or address ranges of the components that want to receive the messages in case of a value change.

Consequences

1. Sensor and actuator classes have a common interface.
2. Class attributes can only be accessed through messages, and the class decides whether or not to accept the message. For example, if a value of an actuator is set above a maximum value, then the actuator class may not accept the message, or it might use a default maximum value.
3. The complexity of the system is potentially reduced because of the uniformity of interfaces for actuators and sensors.

The requirements pattern description might also provide references to other related requirements and design patterns.

7.5 Requirements Modeling for WebApps[14]

Web developers are often skeptical when the idea of requirements analysis for WebApps is suggested. "After all," they argue, "the Web development process must be agile, and analysis is time consuming. It'll slow us down just when we need to be designing and building the WebApp."

Requirements analysis does take time, but solving the wrong problem takes even more time. The question for every WebApp developer is simple—are you sure you understand the requirements of the problem? If the answer is an unequivocal "yes," then it may be possible to skip requirements modeling, but if the answer is "no," then requirements modeling should be performed.

7.5.1 How Much Analysis Is Enough?

The degree to which requirements modeling for WebApps is emphasized depends on the following factors:

- Size and complexity of WebApp increment.
- Number of stakeholders (analysis can help to identify conflicting requirements coming from different sources).
- Size of the WebApp team.
- Degree to which members of the WebApp team have worked together before (analysis can help develop a common understanding of the project).
- Degree to which the organization's success is directly dependent on the success of the WebApp.

The converse of the preceding points is that as the project becomes smaller, the number of stakeholders fewer, the development team more cohesive, and the application less critical, it is reasonable to apply a more lightweight analysis approach.

Although it is a good idea to analyze the problem *before* beginning design, it is not true that *all* analysis must precede *all* design. In fact, the design of a specific part of the WebApp only demands an analysis of those requirements that affect only that part of the WebApp. As an example from *SafeHome*, you could validly design the overall website aesthetics (layouts, color schemes, etc.) without having analyzed the functional requirements for e-commerce capabilities. You only need to analyze that part of the problem that is relevant to the design work for the increment to be delivered.

14 This section has been adapted from Pressman and Lowe [Pre08] with permission.

7.5.2 Requirements Modeling Input

An agile version of the generic software process discussed in Chapter 2 can be applied when WebApps are engineered. The process incorporates a communication activity that identifies stakeholders and user categories, the business context, defined informational and applicative goals, general WebApp requirements, and usage scenarios—information that becomes input to requirements modeling. This information is represented in the form of natural language descriptions, rough outlines, sketches, and other informal representations.

Analysis takes this information, structures it using a formally defined representation scheme (where appropriate), and then produces more rigorous models as an output. The requirements model provides a detailed indication of the true structure of the problem and provides insight into the shape of the solution.

The *SafeHome* **ACS-DCV** (camera surveillance) function was introduced in Chapter 6. When it was introduced, this function seemed relatively clear and was described in some detail as part of a use case (Section 6.2.1). However, a reexamination of the use case might uncover information that is missing, ambiguous, or unclear.

Some aspects of this missing information would naturally emerge during the design. Examples might include the specific layout of the function buttons, their aesthetic look and feel, the size of snapshot views, the placement of camera views and the house floor plan, or even minutiae such as the maximum and minimum length of passwords. Some of these aspects are design decisions (such as the layout of the buttons) and others are requirements (such as the length of the passwords) that don't fundamentally influence early design work.

But some missing information might actually influence the overall design itself and relate more to an actual understanding of the requirements. For example:

Q_1: What output video resolution is provided by *SafeHome* cameras?

Q_2: What occurs if an alarm condition is encountered while the camera is being monitored?

Q_3: How does the system handle cameras that can be panned and zoomed?

Q_4: What information should be provided along with the camera view? (For example, location? time/date? last previous access?)

None of these questions were identified or considered in the initial development of the use case, and yet, the answers could have a substantial effect on different aspects of the design.

Therefore, it is reasonable to conclude that although the communication activity provides a good foundation for understanding, requirements analysis refines this understanding by providing additional interpretation. As the problem structure is delineated as part of the requirements model, questions invariably arise. It is these questions that fill in the gaps—or in some cases, actually help us to find the gaps in the first place.

To summarize, the inputs to the requirements model will be the information collected during the communication activity—anything from an informal e-mail to a detailed project brief complete with comprehensive usage scenarios and product specifications.

7.5.3 Requirements Modeling Output

Requirements analysis provides a disciplined mechanism for representing and evaluating WebApp content and function, the modes of interaction that users will encounter, and the environment and infrastructure in which the WebApp resides.

Each of these characteristics can be represented as a set of models that allow the WebApp requirements to be analyzed in a structured manner. While the specific models depend largely upon the nature of the WebApp, there are five main classes of models:

- **Content model**—identifies the full spectrum of content to be provided by the WebApp. Content includes text, graphics and images, video, and audio data.
- **Interaction model**—describes the manner in which users interact with the WebApp.
- **Functional model**—defines the operations that will be applied to WebApp content and describes other processing functions that are independent of content but necessary to the end user.
- **Navigation model**—defines the overall navigation strategy for the WebApp.
- **Configuration model**—describes the environment and infrastructure in which the WebApp resides.

You can develop each of these models using a representation scheme (often called a "language") that allows its intent and structure to be communicated and evaluated easily among members of the Web engineering team and other stakeholders. As a consequence, a list of key issues (e.g., errors, omissions, inconsistencies, suggestions for enhancement or modification, points of clarification) are identified and acted upon.

7.5.4 Content Model for WebApps

The content model contains structural elements that provide an important view of content requirements for a WebApp. These structural elements encompass content objects and all analysis classes—user-visible entities that are created or manipulated as a user interacts with the WebApp.[15]

Content can be developed prior to the implementation of the WebApp, while the WebApp is being built, or long after the WebApp is operational. In every case, it is

15 Analysis classes were discussed in Chapter 6.

incorporated via navigational reference into the overall WebApp structure. A *content object* might be a textual description of a product, an article describing a news event, an action photograph taken at a sporting event, a user's response on a discussion forum, an animated representation of a corporate logo, a short video of a speech, or an audio overlay for a collection of presentation slides. The content objects might be stored as separate files, embedded directly into Web pages, or obtained dynamically from a database. In other words, a content object is any item of cohesive information that is to be presented to an end user.

Content objects can be determined directly from use cases by examining the scenario description for direct and indirect references to content. For example, a WebApp that supports *SafeHome* is established at **SafeHomeAssured.com.** A use case, *Purchasing Select SafeHome Components,* describes the scenario required to purchase a *SafeHome* component and contains the sentence:

> I will be able to get descriptive and pricing information for each product component.

The content model must be capable of describing the content object **Component.** In many instances, a simple list of content objects, coupled with a brief description of each object, is sufficient to define the requirements for content that must be designed and implemented. However, in some cases, the content model may benefit from a richer analysis that graphically illustrates the relationships among content objects and/or the hierarchy of content maintained by a WebApp.

For example, consider the *data tree* [Sri01] created for a **SafeHomeAssured.com** component shown in Figure 7.10. The tree represents a hierarchy of information that is used to describe a component. Simple or composite data items (one or more data

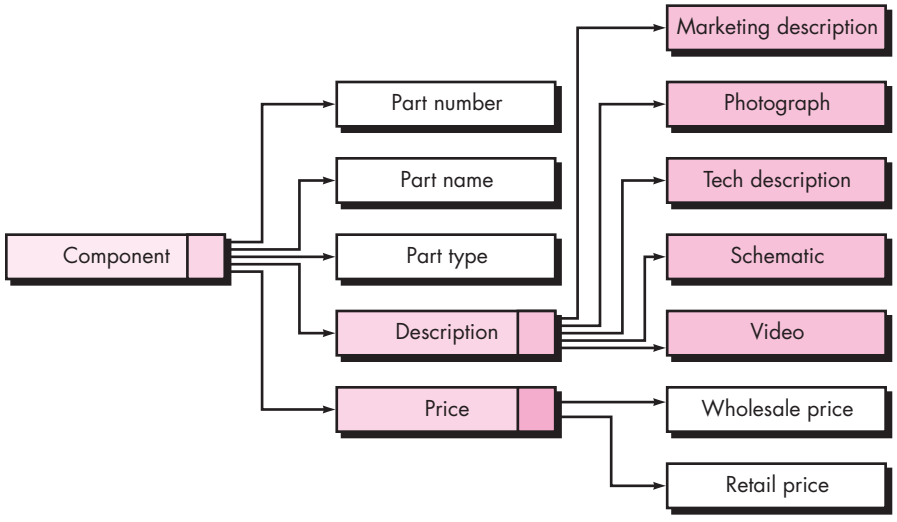


FIGURE 7.10
Data tree for a **SafeHomeAssured.com** component

values) are represented as unshaded rectangles. Content objects are represented as shaded rectangles. In the figure, **description** is defined by five content objects (the shaded rectangles). In some cases, one or more of these objects would be further refined as the data tree expands.

A data tree can be created for any content that is composed of multiple content objects and data items. The data tree is developed in an effort to define hierarchical relationships among content objects and to provide a means for reviewing content so that omissions and inconsistencies are uncovered before design commences. In addition, the data tree serves as the basis for content design.

7.5.5 Interaction Model for WebApps

The vast majority of WebApps enable a "conversation" between an end user and application functionality, content, and behavior. This conversation can be described using an *interaction* model that can be composed of one or more of the following elements: (1) use cases, (2) sequence diagrams, (3) state diagrams,[16] and/or (4) user interface prototypes.

In many instances, a set of use cases is sufficient to describe the interaction at an analysis level (further refinement and detail will be introduced during design). However, when the sequence of interaction is complex and involves multiple analysis classes or many tasks, it is sometimes worthwhile to depict it using a more rigorous diagrammatic form.

The layout of the user interface, the content it presents, the interaction mechanisms it implements, and the overall aesthetic of the user-WebApp connections have much to do with user satisfaction and the overall success of the WebApp. Although it can be argued that the creation of a user interface prototype is a design activity, it is a good idea to perform it during the creation of the analysis model. The sooner that a physical representation of a user interface can be reviewed, the higher the likelihood that end users will get what they want. The design of user interfaces is discussed in detail in Chapter 11.

Because WebApp construction tools are plentiful, relatively inexpensive, and functionally powerful, it is best to create the interface prototype using such tools. The prototype should implement the major navigational links and represent the overall screen layout in much the same way that it will be constructed. For example, if five major system functions are to be provided to the end user, the prototype should represent them as the user will see them upon first entering the WebApp. Will graphical links be provided? Where will the navigation menu be displayed? What other information will the user see? Questions like these should be answered by the prototype.

16 Sequence diagrams and state diagrams are modeled using UML notation. State diagrams are described in Section 7.3. See Appendix 1 for additional detail.

7.5.6 Functional Model for WebApps

Many WebApps deliver a broad array of computational and manipulative functions that can be associated directly with content (either using it or producing it) and that are often a major goal of user-WebApp interaction. For this reason, functional requirements must be analyzed, and when necessary, modeled.

The *functional model* addresses two processing elements of the WebApp, each representing a different level of procedural abstraction: (1) user-observable functionality that is delivered by the WebApp to end users, and (2) the operations contained within analysis classes that implement behaviors associated with the class.

User-observable functionality encompasses any processing functions that are initiated directly by the user. For example, a financial WebApp might implement a variety of financial functions (e.g., a college tuition savings calculator or a retirement savings calculator). These functions may actually be implemented using operations within analysis classes, but from the point of view of the end user, the function (more correctly, the data provided by the function) is the visible outcome.

At a lower level of procedural abstraction, the requirements model describes the processing to be performed by analysis class operations. These operations manipulate class attributes and are involved as classes collaborate with one another to accomplish some required behavior.

Regardless of the level of procedural abstraction, the UML activity diagram can be used to represent processing details. At the analysis level, activity diagrams should be used only where the functionality is relatively complex. Much of the complexity of many WebApps occurs not in the functionality provided, but rather with the nature of the information that can be accessed and the ways in which this can be manipulated.

An example of relatively complex functionality for **SafeHomeAssured.com** is addressed by a use case entitled *Get recommendations for sensor layout for my space.* The user has already developed a layout for the space to be monitored, and in this use case, selects that layout and requests recommended locations for sensors within the layout. **SafeHomeAssured.com** responds with a graphical representation of the layout with additional information on the recommended locations for sensors. The interaction is quite simple, the content is somewhat more complex, but the underlying functionality it very sophisticated. The system must undertake a relatively complex analysis of the floor layout in order to determine the optimal set of sensors. It must examine room dimensions, the location of doors and windows, and coordinate these with sensor capabilities and specifications. No small task! A set of activity diagrams can be used to describe processing for this use case.

The second example is the use case *Control cameras.* In this use case, the interaction is relatively simple, but there is the potential for complex functionality, given that this "simple" operation requires complex communication with devices located remotely and accessible across the Internet. A further possible complication relates

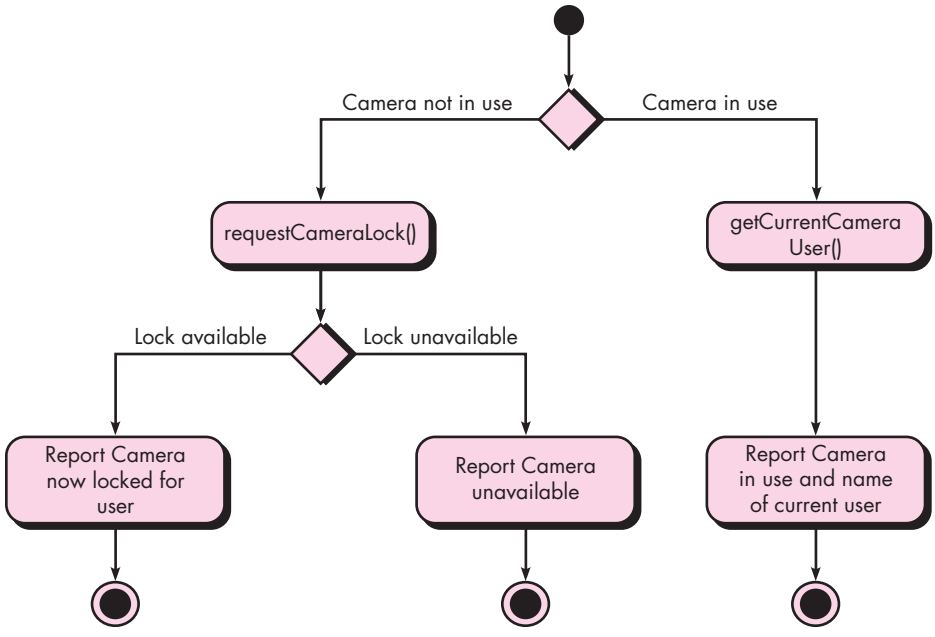


FIGURE 7.11
Activity diagram for the *takeControlOf-Camera()* operation

to negotiation of control when multiple authorized people attempt to monitor and/or control a single sensor at the same time.

Figure 7.11 depicts an activity diagram for the *takeControlOfCamera()* operation that is part of the **Camera** analysis class used within the *Control cameras* use case. It should be noted that two additional operations are invoked with the procedural flow: *requestCameraLock(),* which tries to lock the camera for this user, and *getCurrentCameraUser(),* which retrieves the name of the user who is currently controlling the camera. The construction details indicating how these operations are invoked and the interface details for each operation are not considered until WebApp design commences.

7.5.7 Configuration Models for WebApps

In some cases, the configuration model is nothing more than a list of server-side and client-side attributes. However, for more complex WebApps, a variety of configuration complexities (e.g., distributing load among multiple servers, caching architectures, remote databases, multiple servers serving various objects on the same Web page) may have an impact on analysis and design. The UML *deployment diagram* can be used in situations in which complex configuration architectures must be considered.

For **SafeHomeAssured.com** the public content and functionality should be specified to be accessible across all major Web clients (i.e., those with more than

1 percent market share or greater[17]). Conversely, it may be acceptable to restrict the more complex control and monitoring functionality (which is only accessible to **Homeowner** users) to a smaller set of clients. The configuration model for **SafeHomeAssured.com** will also specify interoperability with existing product databases and monitoring applications.

7.5.8 Navigation Modeling

Navigation modeling considers how each user category will navigate from one WebApp element (e.g., content object) to another. The mechanics of navigation are defined as part of design. At this stage, you should focus on overall navigation requirements. The following questions should be considered:

- Should certain elements be easier to reach (require fewer navigation steps) than others? What is the priority for presentation?
- Should certain elements be emphasized to force users to navigate in their direction?
- How should navigation errors be handled?
- Should navigation to related groups of elements be given priority over navigation to a specific element?
- Should navigation be accomplished via links, via search-based access, or by some other means?
- Should certain elements be presented to users based on the context of previous navigation actions?
- Should a navigation log be maintained for users?
- Should a full navigation map or menu (as opposed to a single "back" link or directed pointer) be available at every point in a user's interaction?
- Should navigation design be driven by the most commonly expected user behaviors or by the perceived importance of the defined WebApp elements?
- Can a user "store" his previous navigation through the WebApp to expedite future usage?
- For which user category should optimal navigation be designed?
- How should links external to the WebApp be handled? Overlaying the existing browser window? As a new browser window? As a separate frame?

These and many other questions should be asked and answered as part of navigation analysis.

17 Determining market share for browsers is notoriously problematic and varies depending on which survey is used. Nevertheless, at the time of writing, Internet Explorer and Firefox are the only browsers that were reported in excess of 30 percent, and Mozilla, Opera, and Safari the only other ones consistently above 1 percent.

You and other stakeholders must also determine overall requirements for navigation. For example, will a "site map" be provided to give users an overview of the entire WebApp structure? Can a user take a "guided tour" that will highlight the most important elements (content objects and functions) that are available? Will a user be able to access content objects or functions based on defined attributes of those elements (e.g., a user might want to access all photographs of a specific building or all functions that allow computation of weight)?

7.6 Summary

Flow-oriented models focus on the flow of data objects as they are transformed by processing functions. Derived from structured analysis, flow-oriented models use the data flow diagram, a modeling notation that depicts how input is transformed into output as data objects move through a system. Each software function that transforms data is described by a process specification or narrative. In addition to data flow, this modeling element also depicts control flow—a representation that illustrates how events affect the behavior of a system.

Behavioral modeling depicts dynamic behavior. The behavioral model uses input from scenario-based, flow-oriented, and class-based elements to represent the states of analysis classes and the system as a whole. To accomplish this, states are identified, the events that cause a class (or the system) to make a transition from one state to another are defined, and the actions that occur as transition is accomplished are also identified. State diagrams and sequence diagrams are the notation used for behavioral modeling.

Analysis patterns enable a software engineer to use existing domain knowledge to facilitate the creation of a requirements model. An analysis pattern describes a specific software feature or function that can be described by a coherent set of use cases. It specifies the intent of the pattern, the motivation for its use, constraints that limit its use, its applicability in various problem domains, the overall structure of the pattern, its behavior and collaborations, and other supplementary information.

Requirements modeling for WebApps can use most, if not all, of the modeling elements discussed in this book. However, these elements are applied within a set of specialized models that address content, interaction, function, navigation, and the client-server configuration in which the WebApp resides.

Problems and Points to Ponder

7.1. What is the fundamental difference between the structured analysis and object-oriented strategies for requirements analysis?

7.2. In a data flow diagram, does an arrow represent a flow of control or something else?

7.3. What is "information flow continuity" and how is it applied as a data flow diagram is refined?

7.4. How is a grammatical parse used in the creation of a DFD?

7.5. What is a control specification?

7.6. Are a PSPEC and a use case the same thing? If not, explain the differences.

7.7. There are two different types of "states" that behavioral models can represent. What are they?

7.8. How does a sequence diagram differ from a state diagram. How are they similar?

7.9. Suggest three requirements patterns for a modern mobile phone and write a brief description of each. Could these patterns be used for other devices. Provide an example.

7.10. Select one of the patterns you developed in Problem 7.9 and develop a reasonably complete pattern description similar in content and style to the one presented in Section 7.4.2.

7.11. How much analysis modeling do you think would be required for **SafeHomeAssured.com?** Would each of the model types described in Section 7.5.3 be required?

7.12. What is the purpose of the interaction model for a WebApp?

7.13. It could be argued that a WebApp functional model should be delayed until design. Present pros and cons for this argument.

7.14. What is the purpose of a configuration model?

7.15. How does the navigation model differ from the interaction model?

Further Readings and Information Sources

Dozens of books have been published on structured analysis. All cover the subject adequately, but only a few do a truly excellent job. DeMarco and Plauger (*Structured Analysis and System Specification,* Pearson, 1985) is a classic that remains a good introduction to the basic notation. Books by Kendall and Kendall (*Systems Analysis and Design*, 5th ed., Prentice-Hall, 2002), Hoffer et al. (*Modern Systems Analysis and Design,* Addison-Wesley, 3d ed., 2001), Davis and Yen (*The Information System Consultant's Handbook: Systems Analysis and Design,* CRC Press, 1998), and Modell (*A Professional's Guide to Systems Analysis,* 2d ed., McGraw-Hill, 1996) are worthwhile references. Yourdon's book (*Modern Structured Analysis,* Yourdon-Press, 1989) on the subject remains among the most comprehensive coverage published to date.

Behavioral modeling presents an important dynamic view of system behavior. Books by Wagner and his colleagues (*Modeling Software with Finite State Machines: A Practical Approach,* Auerbach, 2006) and Boerger and Staerk (*Abstract State Machines,* Springer, 2003) present thorough discussion of state diagrams and other behavioral representations.

The majority of books written about software patterns focus on software design. However, books by Evans (*Domain-Driven Design,* Addison-Wesley, 2003) and Fowler ([Fow03] and [Fow97]) address analysis patterns specifically.

An in-depth treatment of analysis modeling for WebApps is presented by Pressman and Lowe [Pre08]. Papers contained within an anthology edited by Murugesan and Desphande (*Web Engineering: Managing Diversity and Complexity of Web Application Development,* Springer, 2001) treat various aspects of WebApp requirements. In addition, the annual *Proceedings of the International Conference on Web Engineering* regularly addresses requirements modeling issues.

A wide variety of information sources on requirements modeling are available on the Internet. An up-to-date list of World Wide Web references that are relevant to analysis modeling can be found at the SEPA website: **www.mhhe.com/engcs/compsci/pressman/professional/olc/ser.htm**.